

# BetterSearch

Semantic content search: retrieval by **meaning** rather than by shared words.

Search a content library for `Norman conquest` and get back articles about *1066*, the *Battle of Hastings* and the *Domesday Book* — none of which contain the phrase you typed.

Here is the same idea in a library catalogue. One reader, one phrasing, the same 74 records, and the only difference is which button is pressed.

## Catalogue search — 0 results.

 Northfield Libraries

[My account](#) [Events](#) [Help](#)

RS

 R. Sandiford

Full catalogue ▾

Search

Catalogue search

Search by meaning

Try: [learning to be present](#) [gentle crime novels, nothing too gory](#) [grewal](#) [getting my toddler to eat vegetables](#) [books to help me sleep better](#)

Home > Search results

Anywhere: learning to be present

Matched by **keywords** — records containing the words you typed.

Filter results

No results

< Previous    Next >

Nothing in the catalogue matched those words.

The catalogue holds 74 items. A keyword search can only return a record that literally contains what you typed — if a reader's wording differs from the catalogue's, the item cannot be found at all.

Switch to **Search by meaning** to see what is actually here.

## Search by meaning — 6 results.

 **Northfield Libraries** [My account](#) [Events](#) [Help](#) RS R. Sandiford

Full catalogue Search

Catalogue search Search by meaning

Try: [learning to be present](#) [gentle crime novels, nothing too gory](#) [grewal](#) [getting my toddler to eat vegetables](#) [books to help me sleep better](#)

Home > Search results

**Anywhere: learning to be present**

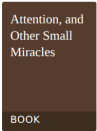
Matched by **meaning** — records about this subject, whatever words they use.

**Filter results** Showing 1 – 6 of 6 < Previous Page 1 Next >

**Availability** ▾  
☐ Available now 6  
☐ Available online 1

**Format** ▾  
☐ Book 4  
☐ Large print 1  
☐ eAudiobook 1

**Fiction / Non-fiction** ▾  
☐ Non-fiction 6

**Attention, and Other Small Miracles**  
Verhoeven, Sanne  
Bellwether, 2017  
*No summary in this record.*  
Contemplation Psychology  
Place reservation Available: 2 Copies: 4 · Fenwick Road

**Living Lightly**

Not one of those six records contains the word *learning* or the word *present*. Four of the six have no summary at all — nothing but a title, an author and a subject heading.

## Why this exists

Keyword search (BM25, Postgres full-text, Elasticsearch's default) ranks by term overlap. If the user's words are not in the document, the document cannot be found — however obviously relevant it is. That is the entire failure mode this proof of concept addresses.

Semantic search embeds text as vectors positioned by meaning, so "Norman conquest" lands near "the invading army of Duke William" without sharing a single content word.

## Install and run

```
pip install -e ".[dev,local,api]" # local backend: no API key needed
bettersearch ingest --corpus data/corpus_seed.json
bettersearch compare "Norman conquest" # the demonstration
bettersearch evaluate --per-query # the measurement
```

Then for the side-by-side UI:

```
uvicorn api.main:app --reload # http://127.0.0.1:8000
```

Nothing above needs an API key, a database, or network access after the first model download. A fresh clone has **no index** — `.bettersearch/` is gitignored, so `ingest` must run before any search.

For deployment, expected outputs and troubleshooting, see [RUNBOOK.md](#).

## The catalogue prototype

The screenshots above come from a working page, not a mockup. It is the same library, the same index and the same `Searcher` — a catalogue-shaped front end over it, for showing the idea to people who will never read an nDCG table.

```
export BETTERSEARCH_INDEX_PATH=.bettersearch/catalogue
bettersearch ingest --corpus data/catalogue_library.json
uvicorn api.main:app --reload      # http://127.0.0.1:8000/catalogue
```

Three queries are worth trying, in this order:

1. `learning to be present` — the pair above. Catalogue search cannot return a record that does not contain the words; meaning-based search returns six.
2. `gentle crime novels, nothing too gory` — a request phrased the way a reader actually asks. Keyword search finds three records and gets the point exactly backwards: it leads with *Nine Grams*, an organised-crime thriller, and picks up a book on spiritual life called *Nothing to Do* because the record contains the word "nothing". Meaning-based search opens with *The Knitting Circle Murders* and has *Tea, Cake and Arsenic* third — but *Nine Grams* still lands second, because "crime novels" is a far stronger signal than "nothing too gory". Better here, not magic.
3. `grewal` — the failure that is easiest to miss, because it looks like it worked:

Anywhere: grewal

Matched by **keywords** — records containing the words you typed.

Filter results

Availability

☐ Available now 9

Format

☐ Book 9

☐ DVD 1

Fiction / Non-fiction

☐ Non-fiction 6

☐ Fiction 4

Publication date

☐ 2010s 3

☐ 2020s 3

☐ 1990s 1

☐ 2000s 1

Location

☐ Ashcombe Branch 3

☐ Fenwick Road 3

☐ Northfield Central 2

☐ Marsden Park 1

☐ Rowley Green 1

Showing 1 - 10 of 10

< Previous Page 1 Next >

1.

[Punjabi book]

**[Punjabi book]**

Grewal, Prem, 1931-1990

UUUU

No summary in this record.

Spiritual life

Place reservation

Available: 2 Copies: 3 · Ashcombe Branch

2.

Nothing to Do

**Nothing to Do**

Grewal, Prem, 1931-1990

Wayfarer Press, UUUU

No summary in this record.

Spiritual life

Place reservation

Available: 3 Copies: 5 · Fenwick Road

3.

The Knitting Circle Murders

**The Knitting Circle Murders**

Grewal, Ravi

Pennyfarthing, 2021

No summary in this record.

Detective and mystery stories

A surname is not an identity. Two unrelated authors share one, so keyword search files a crime novel in among books on spiritual life and looks perfectly healthy doing it. Notice the first record as well: no real title, no summary, no publication date. That record is not a contrived example — a catalogue of any size is full of them, and there is nothing there for a keyword search to match.

The catalogue is a fictional service with invented records, so it can be shown around without passing as a real library. Rebuild it with `python scripts/build_catalogue.py`, and regenerate these screenshots with `python scripts/capture_screenshots.py` against a running server.

## The three modes

| MODE     | HOW IT RANKS                   | GOOD AT                             | BLIND TO                         |
|----------|--------------------------------|-------------------------------------|----------------------------------|
| keyword  | BM25 term overlap              | exact names, dates, figures, jargon | anything phrased differently     |
| semantic | cosine over embeddings         | paraphrase, concepts, questions     | precise tokens it blurs together |
| hybrid   | reciprocal rank fusion of both | most real traffic                   | —                                |

`hybrid` is the mode you would normally reach for — though on this corpus it measurably loses to pure `semantic`; see the results below. Fusion uses **rank**, not raw score: BM25 scores are unbounded and cosine scores sit in  $[-1, 1]$ , so blending the numbers directly needs normalisation constants that must be re-tuned whenever the corpus or model changes. Ranks are comparable by construction.

## Measured results

Run `bettersearch evaluate` to reproduce. 112 documents, 32 hand-labelled queries, `all-MiniLM-L6-v2` embeddings, `numpy` index.

| MODE            | RECALL@5     | MRR@10       | NDCG@10      |
|-----------------|--------------|--------------|--------------|
| keyword         | 0.355        | 0.625        | 0.415        |
| <b>semantic</b> | <b>0.841</b> | <b>0.984</b> | <b>0.890</b> |
| hybrid          | 0.624        | 0.826        | 0.741        |

Semantic retrieval more than doubles nDCG over keyword and wins **30 of the 32 queries** outright. On eight queries — including `Norman conquest`, `why does bread rise`, `father of genetics` and `vaccination` — keyword search scores exactly **0.000**: not a poor ranking, but no relevant document retrieved at any depth.

Two illustrative failures worth seeing in the UI:

- `Norman conquest` → BM25 returns **nothing**. No document contains either word.
- `why does bread rise` → BM25 returns **Inflation**, because that article says "a sustained *rise* in the general price level". Term overlap without meaning.

## Hybrid loses here, and that is a real result

Fusing a strong retriever with a weak one produced a *worse* ranking than the strong one alone (0.741 vs 0.890). That is not a bug in the fusion — it is what RRF does when one input list is mostly empty or noisy, which is the case on an eval set deliberately weighted towards keyword-hostile phrasing.

The honest reading: **ship semantic as the default for this kind of content**, and revisit `hybrid` only against a query mix with more exact-name, code and figure lookups, where BM25 earns its half of the

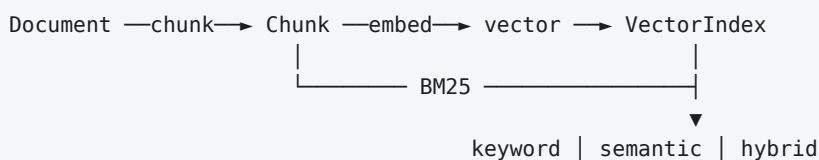
fusion. Sample queries where keyword did contribute: storing energy in a chemical form (keyword 0.871 vs semantic 0.828) and 1066 (0.870 vs 0.876). Run `bettersearch evaluate --per-query` for the full breakdown.

**Read this before quoting the numbers.** The seed corpus and the eval labels were written by the same author, so the absolute values are indicative, not authoritative — the *gap between modes* is the signal. For a non-circular check, build a corpus you did not write:

```
python scripts/fetch_wikipedia.py --count 2000 --out data/corpus_wikipedia.json
bettersearch ingest --corpus data/corpus_wikipedia.json
bettersearch evaluate
```

Note also that the eval set is deliberately weighted towards keyword-hostile phrasing, because searching for words already in the text is the case keyword search already handles. Two control queries (Bayeux Tapestry, evolution by natural selection) use exact terminology, and BM25 is expected to win those.

## Architecture



|                    |                                                      |
|--------------------|------------------------------------------------------|
| src/bettersearch/  |                                                      |
| chunking.py        | paragraph-aware splitting, overlap, content hashing  |
| embeddings/        | EmbeddingProvider protocol + local / OpenAI / Voyage |
| index/             | VectorIndex protocol + numpy / pgvector              |
| keyword.py         | BM25 baseline                                        |
| fusion.py          | reciprocal rank fusion                               |
| search.py          | Searcher – the public entry point                    |
| ingest.py          | incremental ingestion                                |
| evaluate.py        | Recall@5, MRR@10, nDCG@10                            |
| enrichment/        | LLM enrichment of thin catalogue records             |
| cli.py             |                                                      |
| api/main.py        | FastAPI wrapper - holds no retrieval logic           |
| api/catalogue.py   | catalogue-shaped presentation: facets, availability  |
| web/index.html     | developer comparison page                            |
| web/catalogue.html | the catalogue prototype                              |

The library is framework-agnostic. `api/` and `cli.py` are thin callers, and neither is imported by the core.

`web/index.html` is the view for working on retrieval rather than for showing anyone — all three modes on one screen, scored, so a change in chunking or model is visible immediately:

## BetterSearch

The same query, run three ways over the same 112 documents. Keyword search ranks by shared words; semantic search ranks by meaning. Try a query whose wording appears nowhere in the content.

**Search query**

Try: [Norman conquest](#) [why does bread rise](#) [the ship that sank on its maiden voyage](#) [father of genetics](#) [drugs that stop working](#)

Keyword search returned nothing for "Norman conquest". Semantic search found 5 results.

| KEYWORD                                                                   | SEMANTIC                                                                                                                                                                                                       | HYBRID                                                                                                                                                                                                         |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BM25 term overlap. Cannot find a document that does not share your words. | Cosine similarity over embeddings. Matches meaning rather than vocabulary.                                                                                                                                     | Reciprocal rank fusion of the other two columns.                                                                                                                                                               |
| <i>No results — no document shares these terms.</i>                       | <p>0.428 <b>The Battle of Hastings</b></p> <p>On 14 October 1066, an invading army under Duke William of Normandy met the English shield wall on Senlac Hill, a few miles inland from the coast of S...</p>    | <p>0.016 <b>The Battle of Hastings</b></p> <p>On 14 October 1066, an invading army under Duke William of Normandy met the English shield wall on Senlac Hill, a few miles inland from the coast of S...</p>    |
|                                                                           | <p>0.367 <b>Harold Godwinson</b></p> <p>The last crowned Anglo-Saxon king of England, Harold was the most powerful nobleman in the kingdom before taking the throne in January 1066. His claim...</p>          | <p>0.016 <b>Harold Godwinson</b></p> <p>The last crowned Anglo-Saxon king of England, Harold was the most powerful nobleman in the kingdom before taking the throne in January 1066. His claim...</p>          |
|                                                                           | <p>0.328 <b>Motte-and-Bailey Castles</b></p> <p>A raised earth mound topped by a wooden tower, with an enclosed courtyard below protected by a ditch and palisade. Cheap, fast to build, and defensibl...</p>  | <p>0.016 <b>Motte-and-Bailey Castles</b></p> <p>A raised earth mound topped by a wooden tower, with an enclosed courtyard below protected by a ditch and palisade. Cheap, fast to build, and defensibl...</p>  |
|                                                                           | <p>0.322 <b>William, Duke of Normandy</b></p> <p>Born illegitimate in Falaise around 1028, William inherited his duchy as a child and spent two decades fighting to hold it. He claimed the English thr...</p> | <p>0.016 <b>William, Duke of Normandy</b></p> <p>Born illegitimate in Falaise around 1028, William inherited his duchy as a child and spent two decades fighting to hold it. He claimed the English thr...</p> |
|                                                                           | <p>0.316 <b>The Battle of Britain</b></p> <p>Between July and October 1940 the Luftwaffe attempted to destroy British air defences as a precondition for invasion. Fighter Command was outnumbered ...</p>     | <p>0.015 <b>The Battle of Britain</b></p> <p>Between July and October 1940 the Luftwaffe attempted to destroy British air defences as a precondition for invasion. Fighter Command was outnumbered ...</p>     |

112 chunks indexed · provider local · model sentence-transformers/all-MiniLM-L6-v2@384 · 384 dims · numpy backend

```
$ bettersearch compare "Norman conquest" # the same thing at the terminal
```

## Configuration

Everything is a `BETTERSEARCH_*` environment variable; no code change is needed to switch provider or storage.

| VARIABLE                             | DEFAULT                                | NOTES                                                                 |
|--------------------------------------|----------------------------------------|-----------------------------------------------------------------------|
| BETTERSEARCH_EMBEDDING_PROVIDER      | local                                  | local   openai   voyage                                               |
| BETTERSEARCH_INDEX                   | numpy                                  | numpy   pgvector                                                      |
| BETTERSEARCH_INDEX_PATH              | .bettersearch/index                    | numpy backend only                                                    |
| BETTERSEARCH_DATABASE_URL            | —                                      | pgvector backend; falls back to DATABASE_URL                          |
| BETTERSEARCH_EMBEDDING_DIMENSIONS    | 512                                    | Matryoshka width for API providers                                    |
| BETTERSEARCH_CHUNK_TOKENS            | 450                                    | target chunk size                                                     |
| BETTERSEARCH_CHUNK_OVERLAP           | 60                                     | overlap between chunks                                                |
| BETTERSEARCH_LOCAL_MODEL             | sentence-transformers/all-MiniLM-L6-v2 | any sentence-transformers model                                       |
| BETTERSEARCH_LOCAL_TRUST_REMOTE_CODE | 0                                      | required by some long-context encoders; runs code from the model repo |

API keys are read from `OPENAI_API_KEY` / `VOYAGE_API_KEY` / `ANTHROPIC_API_KEY` at the point of use and never stored by this package. Only enrichment needs an Anthropic key; indexing and search do not.

## What actually breaks at scale

The embedding model is not the bottleneck people expect. Four other things are. Three have a specific countermeasure in the code; the fourth was found by measuring a real 4,000-record catalogue and is not fixed yet.

### 1. Changing embedding model invalidates the whole index

Vectors from two models occupy different spaces. Comparing them returns confident nonsense — the worst kind of bug, because nothing errors.

Every vector is stored with its `model_id` and `dims`, and both index backends raise `ModelMismatchError` rather than answer a query embedded by a different model. That column is also what makes a *rolling* re-embed possible: without it, changing model means taking search offline.

### 2. Re-ingesting must not re-embed everything

Every chunk carries a `content_hash` over its embed text. Ingestion skips hashes already indexed, so a content update costs only the changed chunks. Re-run `bettersearch ingest` twice — the second run reports zero embedded.

### 3. Vector storage is the real wall

At 1M chunks × 1536 dims × 4 bytes you are holding **5.72 GB** of raw vectors before index overhead. Three multiplicative fixes, all implemented:

| LEVER                              | EFFECT                |
|------------------------------------|-----------------------|
| Matryoshka truncation (1536 → 512) | 3× smaller            |
| <code>halfvec</code> fp16 storage  | 2× smaller            |
| HNSW rather than flat scan         | sub-linear query time |

Together that is 6x: **5.72 GB** → **0.95 GB**, the difference between needing a large managed Postgres instance and fitting comfortably on a small one. Note the self-hosted MiniLM default is smaller still — 384 dims at fp16 is 0.72 GB for 1M chunks — so staying local sidesteps most of this rather than creating it.

#### 4. An absolute relevance threshold does not survive a real corpus

`api/catalogue.py` cut meaning-based results at a fixed `RELEVANCE_FLOOR = 0.15`, tuned by eye on 74 records. Measured against 4,000 real ones with the same 41 queries, it stopped being a threshold at all:

| RESULTS RETURNED PER QUERY               | MIN | MEDIAN     | P90   | MAX          |
|------------------------------------------|-----|------------|-------|--------------|
| <b>absolute floor, 0.15</b>              | 116 | <b>763</b> | 3,262 | <b>3,665</b> |
| relative: $\geq 0.90 \times \text{best}$ | 1   | 4          | 14    | 20           |
| relative: $\geq 0.85 \times \text{best}$ | 2   | 9          | 27    | 41           |
| largest-gap cut                          | 1   | 2          | 6     | 24           |

`books` like `Agatha Christie` returns **3,665 of 4,000 records** above the floor — 92% of the catalogue, presented to a reader as "3,665 results". The count is worse than useless: it is confidently wrong.

The reason is that a fixed cosine threshold admits a roughly constant *fraction* of any corpus, not a constant number — 31% of the 74-record catalogue, 19% of the 4,000-record one — so the absolute count grows with the collection while the threshold looks unchanged. It scales exactly the wrong way, and nothing errors. This is the same brittleness that showed up when renaming one author moved a headline query from 8 results to 6: a single global constant cannot carry this.

**The obvious fix is a relative cut**, and  $\geq 0.85 \times \text{best}$  is the plausible candidate on these numbers. It was not applied, because which cut is *right* is a question about relevance, and that needs the judged eval set rather than a count that looks sensible. Sane-looking output is not evidence — and when the judgements arrived they picked something else entirely. See *Settling the relevance floor* below.

#### 5. Scaling while staying self-hosted

Self-hosted embeddings are the default path here: no per-token cost, no vendor dependency, and no content leaving your infrastructure — which matters more than the money if the corpus is ever commercially or personally sensitive.

**Measure before you optimise.** On this content the much-discussed 256-token cap costs exactly nothing:

```
$ # chunk length as all-MiniLM-L6-v2 itself tokenises it
  seed corpus (112 chunks)    min 75   median 87   p90 98   max 115
  catalogue   (74 chunks)    min 17   median 29   p90 49   max 57
  chunks exceeding the 256 cap: 0 (0%)
```

Measured with the model's own tokeniser, not the chunker's word-based estimate — the cap is the model's, so the model's count is the one that decides. Both corpora are short enough that no chunk reaches even half the cap, let alone the 450-token chunk target, so MiniLM truncates nothing. `bettersearch ingest` reports `truncated_chunks` precisely so this is a measurement rather than an assumption — check it against *your* content before changing anything.

**The cap is architectural, not a compute limit.** MiniLM stops at 256 tokens because its positional embeddings end there. More CPU or a bigger GPU makes it faster, never able to read more. If your chunks *do* exceed it, there are two fixes and only one of them costs anything:

1. **Shrink the chunks** — `BETTERSEARCH_CHUNK_TOKENS=200`. Free, no model change. Costs you more chunks (bigger index) and more fragmented context.
2. **Swap the model** — one env var, no code change. Costs compute and RAM.

```
export BETTERSEARCH_LOCAL_MODEL="BAAI/bge-base-en-v1.5"
bettersearch ingest --corpus data/corpus_seed.json # re-embeds under the new model
```

Dimensions and the input limit are read off the loaded model, so the index, the truncation warning and the `model_id` guard all follow automatically.

| MODEL                                       | INPUT | DIMS | SIZE  | NOTES                                                     |
|---------------------------------------------|-------|------|-------|-----------------------------------------------------------|
| <code>all-MiniLM-L6-v2</code>               | 256   | 384  | 80MB  | current default                                           |
| <code>BAAI/bge-small-en-v1.5</code>         | 512   | 384  | 130MB | 2× window, same index size                                |
| <code>BAAI/bge-base-en-v1.5</code>          | 512   | 768  | 440MB | 2× index size                                             |
| <code>intfloat/e5-large-v2</code>           | 512   | 1024 | 1.3GB |                                                           |
| <code>nomic-ai/nomic-embed-text-v1.5</code> | 8192  | 768  | 550MB | needs <code>BETTERSEARCH_LOCAL_TRUST_REMOTE_CODE=1</code> |
| <code>Alibaba-NLP/gte-large-en-v1.5</code>  | 8192  | 1024 | 1.7GB | needs trust-remote-code                                   |
| <code>BAAI/bge-m3</code>                    | 8192  | 1024 | 2.3GB | multilingual                                              |

**Bigger is not automatically better.** Measured on this corpus, the 512-token model scored *worse* than MiniLM (nDCG 0.851 vs 0.890) — because nothing was being truncated, so the longer window bought nothing while the model itself happened to suit short passages less well. Re-run `bettersearch evaluate` after any model change; that is what the eval harness is for.

**Measured CPU throughput** on an ordinary container (no GPU), for a one-off backfill:

| MODEL                          | CHUNKS/SEC (CPU) | 1M CHUNKS  |
|--------------------------------|------------------|------------|
| <code>all-MiniLM-L6-v2</code>  | ~119             | ~2.3 hours |
| <code>bge-small-en-v1.5</code> | ~74              | ~3.8 hours |

A single modern GPU is roughly 20–50× that, turning a backfill into minutes. Note this is a **batch** cost paid once per re-embed; query time is one embedding per search and is negligible either way.

The real steady-state cost of self-hosting is **RAM per web worker** (~600MB resident for MiniLM, more for larger models), not throughput. On a small Render instance that is the constraint that bites first — the usual fix is a separate embedding worker rather than loading the model into every web process.

## If you ever do want a hosted provider

`openai` and `voyage` backends exist behind the same interface and are selected with `BETTERSEARCH_EMBEDDING_PROVIDER`. **Neither has been exercised against a live API** — they are verified only as far as constructing, reporting correct dimensions, refusing to run without a key, and handling empty batches. Treat them as a starting point, not a supported path.

| PROVIDER                                   | 1M CHUNKS (~400M TOKENS) | INPUT LIMIT   |
|--------------------------------------------|--------------------------|---------------|
| <code>openai text-embedding-3-small</code> | ~\$8 (~\$4 batched)      | 8,191 tokens  |
| <code>voyage voyage-3.5-lite</code>        | ~\$8                     | 32,000 tokens |

Voyage additionally supports asymmetric encoding ( `input_type` of `document` vs `query` ), which is a real recall gain and is wired in.

## Measured on 4,000 real catalogue records

Everything above this point was measured on records I wrote. This section is measured on **4,000 real bibliographic records** from Open Library, against 41 queries I drafted and Shen reviewed, with relevance judged by Claude Haiku on a pooled candidate set. Build it with `scripts/fetch_openlibrary.py` and `scripts/build_eval_set.py`.

### The three modes

| MODE            | RECALL@5     | MRR@10       | NDCG@10      |
|-----------------|--------------|--------------|--------------|
| keyword         | 0.205        | 0.609        | 0.429        |
| <b>semantic</b> | <b>0.291</b> | <b>0.744</b> | <b>0.588</b> |
| hybrid          | 0.260        | 0.720        | 0.555        |

**Hybrid still loses, and that kills a hypothesis.** The earlier result — hybrid below pure semantic — was explained away as an artefact of an eval set weighted towards keyword-hostile phrasing, with the prediction that real catalogue traffic, full of exact-name lookups, would be where fusion finally earned its place. It did not. On real records with five deliberate exact-title and exact-author controls, hybrid still sits below semantic. The earlier explanation was wrong, or at least incomplete.

**The eval is not saturated.** MRR@10 is 0.744, not the 1.000 that made the previous query set useless. There is room to improve and room to regress, which is the only condition under which a number means anything.

### Enrichment, tested fairly for the first time

Enrichment on real records, Haiku 4.5, all 4,000:

| ARM             | RECALL@5     | MRR@10       | NDCG@10      |
|-----------------|--------------|--------------|--------------|
| thin records    | 0.291        | 0.744        | 0.588        |
| <b>enriched</b> | <b>0.328</b> | <b>0.787</b> | <b>0.669</b> |

**+0.081 nDCG, a 14% relative gain**, improving 27 of 41 queries. Far more modest than the synthetic corpus suggested, and far more believable. Biggest gains are where a title says nothing about content:

| QUERY                                        | THIN  | ENRICHED     |
|----------------------------------------------|-------|--------------|
| cycling long distances                       | 0.366 | <b>0.747</b> |
| Joy of Cooking                               | 0.631 | <b>1.000</b> |
| a novel where the beekeeper is the detective | 0.307 | <b>0.613</b> |
| an uplifting story after a hard year         | 0.073 | <b>0.344</b> |
| second world war in the pacific              | 0.073 | <b>0.290</b> |

**Enrichment is not free, and the failures are systematic.** Five queries got worse, and the worst is instructive:

| QUERY                          | THIN         | ENRICHED |
|--------------------------------|--------------|----------|
| Sue Monk Kidd                  | <b>1.000</b> | 0.387    |
| what to do about money worries | 1.000        | 0.695    |

Sue Monk Kidd is an exact-author lookup with one matching record. Enrichment surrounds that record with topical prose about bees, race and the American South, and the author's name stops dominating its vector. **Enrichment helps queries about aboutness and hurts queries about identity** — which is an argument for routing by query type, not for enriching everything and hoping.

**It also reintroduces a problem the project had written off.** Enrichment takes the median record from 21 words to 117, and **79 chunks now exceed MiniLM's 256-token cap** where thin records exceeded it zero times. The earlier finding that the cap "costs exactly nothing" was true only of thin records. Two decisions each measured in isolation interact, and Opus enrichment — 74-word synopses against Haiku's 44 — would truncate more.

## Settling the relevance floor

`RELEVANCE_FLOOR = 0.15` was tuned by eye on 74 records. Against real judgements, on the thin 4,000-record arm:

| STRATEGY                                | PRECISION    | RECALL | F1           | MEDIAN RESULTS |
|-----------------------------------------|--------------|--------|--------------|----------------|
| absolute 0.15 (old)                     | <b>0.035</b> | 0.986  | 0.067        | 763            |
| relative $\geq 0.90 \times \text{best}$ | 0.582        | 0.302  | 0.398        | 4              |
| relative $\geq 0.85 \times \text{best}$ | 0.502        | 0.400  | 0.445        | 9              |
| largest-gap cut                         | 0.648        | 0.246  | 0.356        | 2              |
| fixed top-5                             | 0.522        | 0.291  | 0.373        | 5              |
| fixed top-10                            | 0.461        | 0.420  | 0.439        | 10             |
| fixed top-15                            | 0.418        | 0.556  | 0.477        | 15             |
| <b>fixed top-20</b>                     | 0.398        | 0.667  | <b>0.498</b> | 20             |

Precision **0.035**: the floor admits almost everything and calls it a result set. But the fix I proposed from result counts alone — a relative cut — is **not** the winner. Plain top-k beats every threshold on F1, because thresholds buy precision by throwing away recall the pagination would have handled anyway. Judging by counts that "look sensible" would have picked the wrong answer; this is why the fix waited for the judgements.

**Where the evidence stops.** The eval pooled each lane to depth 20, so no record below rank 20 has a judgement and every one of them scores as irrelevant. F1 is still *rising* at k=20 — 0.373, 0.439, 0.477, 0.498 across the sweep — and the fall after it (0.477 at 25, 0.339 at 50) is an artefact of running past the pool, not a peak. So the honest claim is narrow: **20 is the deepest cut this eval can vouch for, and it beats everything shallower.** Whether 30 would be better is unmeasured, and would need a deeper pool to answer.

api/catalogue.py now retrieves SEMANTIC\_TOP\_K = 20 directly instead of retrieving everything and filtering. books like Agatha Christie returns 20 results rather than 3,665. Reproduce the whole table, both arms and the sweep:

```
python scripts/tune_cutoff.py thin=.bettersearch/real \
    enriched=.bettersearch/real-enriched
```

One visible consequence: facet counts in the sidebar now describe the 20 results returned, where before they described a 763-record set the reader could never page to. The keyword lane is untouched and still returns full depth — BM25 stops on its own, since a record sharing no term with the query does not match at all, while cosine similarity scores every record against every query and needs the list ended for it.

### Open: does the top model tier buy better retrieval?

Enrichment above is Haiku 4.5 on all 4,000 records. Opus 5 was run on a 289-record sample before the comparison was abandoned, and on those same records it is plainly the richer writer:

| ON THE SAME 289 RECORDS | HAIKU 4.5 | OPUS 5 |
|-------------------------|-----------|--------|
| recognised: true        | 52%       | 76%    |
| synopsis words          | 39        | 65     |
| topics                  | 7.6       | 13.5   |

An earlier version of this table read 74% / 93% off the first 120 records. Those 120 are all detective and science fiction — the corpus is stored in the order it was fetched, subject by subject — and famous genre novels are the easiest thing in a catalogue for a model to recognise. Widening the sample to 289 drops both models by more than twenty points. The ordering between them survives; the absolute rates did not, and a sample drawn from the top of a subject-ordered file is not a sample.

**This does not answer whether Opus retrieves better, and it was never going to.** A valid Opus arm needs all 4,000 records enriched by Opus: with 289 enriched among 4,000 thin ones, the enriched records gain an advantage purely for being enriched. The sample supports a quality comparison, not a retrieval one. Buying the real answer costs about £37 of Opus enrichment against £7 for Haiku.

**The 4k result argues against assuming richer is better.** Enrichment already regressed five queries by diluting records — Sue Monk Kidd fell 1.000 → 0.387 because topical prose crowded out an exact-author match. Opus writes 68% more synopsis and 78% more topics, so it would dilute harder. Richer could

plausibly retrieve *worse*, and the 79 chunks already pushed past MiniLM's 256-token cap would become more.

The 289 Opus enrichments are committed at `data/enrichment_real_opus_sample.jsonl` so the quality figures above can be checked.

### How much to trust this

- **Judge self-consistency 98%** (98 of 100 re-judged pairs identical). A 50-pair sample is in `data/eval_real_spotcheck.md` for a human to check.
- **Pooling bias.** Records no lane retrieved were never judged, so recall is relative to the pool, not absolute. All three measured lanes contributed — 309 records came only from the enriched lane, and judging before enriching would have scored every one of them as irrelevant.
- **Grades are an LLM's**, validated by sampling, not human ground truth.
- **Queries are author-drafted, reviewer-edited.** Not independently written.
- **Holdings are invented.** Bibliographic data is real; availability, branch and format are not.
- **Two queries score 0.000 on both arms** — `learning to grow vegetables in pots` and `the one about a boy wizard at school`. Reported rather than quietly dropped.

---

## Enrichment for thin catalogue records

The production target is a library catalogue of 100k–1M items holding **thin records only** — title, author, subject headings. Around 15 words per item. That is not enough surface to embed: a catalogue record names a shelf position, while semantic search needs text that names *concepts*.

The enrichment pipeline generates that text with an LLM, and can state concepts the record never contains — turning `Subjects: Great Britain -- History` into prose mentioning the Norman Conquest, William the Conqueror and the Domesday survey. It builds the vocabulary bridge deliberately instead of hoping for it.

### How much does thinness actually cost? (measured)

`scripts/synthesise_catalogue.py` strips the seed corpus down to thin records — title, synthetic author, LCSH-style subject headings, **no body text at all** — so the same 32 labelled queries can be re-run against a realistically impoverished corpus, then enriched and re-run a third time.

| ARM              | CORPUS            | RECALL@5 | MRR@10       | NDCG@10      |
|------------------|-------------------|----------|--------------|--------------|
| Full text        | full article text | 0.841    | 0.984        | 0.890        |
| <b>Baseline</b>  | thin records only | 0.718    | 0.917        | <b>0.789</b> |
| <b>Treatment</b> | enriched records  | 0.836    | <b>1.000</b> | <b>0.894</b> |

Enrichment recovers **104% of the gap** between thin records and full text, and improves 24 of the 32 queries. The per-query view is where the case is actually made — run `python scripts/compare_arms.py ceiling=... thin=... enriched=...`:

| QUERY                                        | FULL TEXT | THIN  | ENRICHED     |
|----------------------------------------------|-----------|-------|--------------|
| planning a landing on a defended coast       | 0.871     | 0.296 | <b>0.626</b> |
| 1066                                         | 0.876     | 0.369 | <b>0.706</b> |
| how electricity gets from a power station... | 1.000     | 0.581 | <b>0.884</b> |
| how plants make food from sunlight           | 0.761     | 0.503 | <b>0.956</b> |
| drugs that stop working                      | 0.920     | 0.693 | <b>1.000</b> |

**Queries collapse when the relevant documents' titles do not name the concept**, and that is exactly what enrichment repairs. 1066 falls by more than half on thin records because *The Domesday Book*, *The Bayeux Tapestry*, *Stamford Bridge* and *Harold Godwinson* never contain the string — enrichment puts it there, and the query recovers to 0.706.

### Four things that keep this honest

**"Full text" is not a ceiling, and enrichment passes it.** 0.894 against 0.890, beating it on 17 of 32 queries. That is not a paradox: an article was written to be read, while enrichment is generated to be *retrieved* — the `questions` field puts "What happened at the Battle of Hastings?" into the embedded text, and query-to-question is a closer shape match than query-to-document. The row is relabelled from "Ceiling" accordingly.

**MRR@10 hit 1.000 — the eval set is saturated.** A relevant record now ranks first for all 32 queries, so this query set can no longer distinguish further improvement. Any next experiment needs harder queries or a larger corpus; do not read 1.000 as headroom.

**Enrichment made 5 of 32 queries worse**, and it is not free. `machine that broke German codes in the war` fell 0.832 → 0.613: adding topical prose can dilute a record that already matched well. The aggregate hides this — check the per-query table before assuming enrichment is a pure win.

**Every record came back recognised: false (0 of 112).** The authors are invented, so the model correctly declined to claim knowledge of the specific works and used grounded expansion throughout — subject headings turned into prose, nothing fabricated. So these numbers measure the **conservative mode only**. The knowledge-synopsis path is still unmeasured, and on a real catalogue of real books it would carry a meaningful share of the corpus.

## Design

Three decisions carry the risk:

**The generated text is a retrieval bridge and is never displayed.** Match against it, then show the real catalogue record. A library is an authority-bearing institution; this one rule demotes a hallucination from a false statement by the library to a mildly bad search result.

**Two modes, chosen per item by the model.** Thin records maximise hallucination exposure — given only a title and a subject heading, the model draws on training knowledge rather than the record. So it first reports whether it actually knows the work:

| RECOGNISED | MODE               | MAY USE                                                          |
|------------|--------------------|------------------------------------------------------------------|
| true       | Knowledge synopsis | Its own knowledge of the work. Rich.                             |
| false      | Grounded expansion | <b>Only the record.</b> Expand subject headings; invent nothing. |

Grounded expansion is the safety floor and is still worth the call — turning authoritative controlled vocabulary into searchable prose is the bridge we want, with no fabrication risk. `bettersearch enrich` reports the split.

**Enrichment is added alongside the record, never instead of it.** The raw record stays indexed so the keyword lane can still match exact author, title and subject-heading terms. This is where `hybrid` should finally beat pure `semantic`, unlike on the article corpus above — real catalogue traffic is full of exact-name lookups, which is what BM25 is for. Re-measure rather than carrying the earlier conclusion across.

**Prompt version is tracked like model id.** A prompt change invalidates every stored enrichment, exactly as an embedding-model change invalidates every vector — except regenerating costs money rather than CPU. Every record stores `prompt_version`, `enrichment_model` and a `source_hash` of the input, so re-runs are incremental and a prompt edit is a costed, staged migration.

## Running it

```
export ANTHROPIC_API_KEY=...
python scripts/synthesise_catalogue.py          # or bring your own catalogue
bettersearch enrich --corpus data/catalogue_thin.json --index
```

**Reproducing the table above needs no API key.** The 112 enrichments from that run are committed at `data/enrichment_seed.jsonl` — copy them into the store and every number is re-derivable offline:

```
mkdir -p .bettersearch && cp data/enrichment_seed.jsonl .bettersearch/enrichment.jsonl
BETTERSEARCH_INDEX_PATH=.bettersearch/arm-ceiling bettersearch ingest --corpus
data/corpus_seed.json
BETTERSEARCH_INDEX_PATH=.bettersearch/arm-thin bettersearch ingest --corpus
data/catalogue_thin.json
BETTERSEARCH_INDEX_PATH=.bettersearch/arm-enriched bettersearch enrich \
--corpus data/catalogue_thin.json --index      # store is warm: no API calls
python scripts/compare_arms.py ceiling=.bettersearch/arm-ceiling \
thin=.bettersearch/arm-thin enriched=.bettersearch/arm-enriched
```

Uses the Batch API by default (half price, results keyed by `custom_id` since they arrive in arbitrary order). Resumable: everything already stored for this prompt/model/source is skipped, so a crash costs only the in-flight batch. `--sync` switches to one request per item for small runs.

## Cost

Thin records invert the usual shape — the record itself is ~26 tokens against a cached system prompt, and output is the bulk — so **synopsis length is the primary lever, not model tier**.

Measured on the 112-record run above (Opus 5, Batch API):

|                |        |                |                   |             |        |
|----------------|--------|----------------|-------------------|-------------|--------|
| uncached input | 6,353  | cache writes   | 84,392            | cache reads | 38,360 |
| output         | 29,353 | mean           | 262 tokens/record |             |        |
| total spend    | \$0.66 | cache hit rate | 31%               |             |        |

**The 31% cache hit rate is the finding worth carrying forward.** A batch submits every request at once, so they race on the system-prompt cache and most *write* it rather than read it. That pushed input to 44% of the bill — not the rounding error "output dominates" implies. At catalogue scale a sustained run keeps the prefix warm and the ratio improves, but this is exactly the silent cost leak `client.py` warns about, so check `cache_read_input_tokens` on any real run before trusting a projection.

Projections below assume caching works properly; at the 31% hit rate measured here the Opus figure is nearer \$5,900 than \$4,000.

| MODEL            | 1M ITEMS (BATCHED) | 100K ITEMS |
|------------------|--------------------|------------|
| Haiku 4.5        | ~\$800             | ~\$80      |
| Sonnet 5         | ~\$1,600           | ~\$160     |
| Opus 5 (default) | ~\$4,000           | ~\$400     |

At 100k the tier barely matters. At 1M it is a real decision — so measure it rather than guessing. Enrich a few hundred items at each tier into the same store (the model is part of the cache key, so tiers coexist) and compare with `scripts/compare_arms.py`. Embedding stays local and free either way.

## Using the pgvector backend

```
docker compose up -d
export
BETTERSEARCH_DATABASE_URL="postgresql://bettersearch:bettersearch@localhost:5433/bettersearch"
export BETTERSEARCH_INDEX=pgvector
bettersearch ingest --corpus data/corpus_seed.json
bettersearch compare "Norman conquest"
```

Results should be identical to the numpy backend. The schema is created on first write and includes a generated `tsvector` column with a GIN index — the query to switch to when the corpus outgrows loading every chunk into memory.

## HTTP API

| ENDPOINT      | PURPOSE                                                                          |
|---------------|----------------------------------------------------------------------------------|
| GET /health   | provider, backend, chunk count, <code>model_id</code>                            |
| POST /search  | { <code>query</code> , <code>mode</code> , <code>top_k</code> } → ranked results |
| POST /compare | one query, all three modes                                                       |

Responses follow `{"status": "success", "data": {...}}`.

## Dropping this into a Django/DRF project

The core library has no framework dependency, so the whole integration is one view:

```
# search/views.py
from bettersearch import Searcher
from rest_framework.response import Response
from rest_framework.views import APIView

_searcher = Searcher()          # module-level: loads the model once

class SemanticSearchView(APIView):
    def post(self, request):
        query = request.data.get("query", "").strip()
        if not query:
            return Response({"status": "error", "message": "query is required"}, status=400)
        response = _searcher.search(
            query,
            mode=request.data.get("mode", "hybrid"),
            top_k=int(request.data.get("top_k", 10)),
        )
        return Response({"status": "success", "data": response.to_dict()})
```

Point `BETTERSEARCH_DATABASE_URL` at the existing database and the chunk table lives alongside the rest of the schema. Embedding work belongs in a Celery task, not the request cycle.

---

## Scope

Deliberately **not** included: LLM answer generation, query expansion, a reranker, authentication, and multi-tenant scoping. This PoC answers one question — does retrieval by meaning beat retrieval by keyword on this content — and each of those additions would obscure the answer.

## Tests

```
pytest
```

Covers chunk boundaries and overlap, hash stability, the incremental-ingest skip, model-mismatch refusal, BM25 ranking, RRF ordering, numpy top-k against hand-computed values, and the full enrichment pipeline — resumability, prompt invalidation, per-model isolation, and out-of-order batch results. The tests use a deterministic fake embedding provider and a fake enrichment client, so they need no model, no network and no API key.